

FinLake

BFSI Data Platform — Portfolio Project

100% Free & Open Source · Local Docker Stack · Zero Cloud Cost · 12 Months to Build

WHAT IS FINLAKE

The project that survives every senior DE interview

FinLake is a production-grade, end-to-end financial data platform built entirely with open-source tools running on your local machine via Docker Compose. It ingests real NSE/BSE stock market data using the free **yfinance** Python library, processes it through a full **Medallion architecture** (Bronze → Silver → Gold) using **Apache Iceberg** tables on **MinIO** storage, orchestrates everything with **Apache Airflow**, adds real-time simulation via **Apache Kafka + PySpark Structured Streaming**, enforces data quality with **Great Expectations**, and serves analytics via **Apache Superset** dashboards. Every component mirrors what Razorpay, CRED, and PhonePe actually run in production. One **docker-compose up** starts everything.

FINAL GOAL — What exists at the end of 12 months

OUTCOME	WHAT IT LOOKS LIKE
GitHub Repo	- Public repository with 6+ months of git history showing consistent work - README with architecture diagram, one-command setup: docker-compose up - Architecture Decision Records (ADRs) explaining every tool choice
Working System	- Full pipeline: yfinance API → Iceberg Bronze → dbt Silver/Gold → Superset dashboard - Kafka streaming writes Bronze Iceberg table alongside batch Airflow DAG - Great Expectations DQ checks at Bronze and Silver, pipeline halts on failure
Published Writing	2 blog posts on Hashnode/Substack documenting real problems you hit and solved - dbt docs auto-generated and committed to repo 5 ADR markdown files — one per major architectural decision
Interview-Ready	- Can demo the live system in a screen share during any DE interview - Can explain every design decision when the interviewer pushes back - Can walk through a production incident: Kafka consumer lag, DQ failure, Iceberg compaction
Cert Coverage	- Covers ~80% of Databricks Certified Data Engineer Associate exam content - Airflow DAG patterns cover most of the Apache Airflow Provider exam - dbt project covers dbt Analytics Engineer certification content

SYSTEM REQUIREMENTS

What your machine needs to run 12 Docker services

RESOURCE	REQUIREMENT
RAM	16 GB recommended. 8 GB works if you run Phase 1 subset (Airflow + MinIO + Spark only, add services progressively).
Disk Space	50 GB free minimum. Docker images ~15 GB. MinIO data + Iceberg warehouses ~20 GB by Phase 6.
CPU	4 cores minimum. PySpark local mode uses all available cores — more cores = faster transforms.
OS	Ubuntu 22.04+ or macOS 13+. Windows works with WSL2. All services run in Docker — OS matters less than RAM/disk.
Software	Docker Engine 24+ and Docker Compose v2. Both free from docker.com. Git for version control.
Network	Internet only for initial Docker image pulls and yfinance API calls. Everything else runs locally after setup.

THE VIBE CODING RISK: You can use AI to write boilerplate — that is fine and expected. But interviewers at CRED/Razorpay/PhonePe WILL ask exactly three things this project cannot fake: (1) **Why Iceberg and not Delta Lake?** (2) **What happens when your Kafka consumer falls behind — how do you handle the lag?** (3) **Walk me through a DQ failure in your pipeline and how the system recovered.** For every component you build, read the official docs for one hour. That is the minimum viable understanding that separates a candidate who built the project from one who vibe-coded it.

SOURCE LAYER — Real Financial Market Data

[Yahoo Finance via yfinance (free, MIT license, no API key required)] [NSE / BSE: OHLCV prices, volume, splits, dividends for 20 Nifty-50 stocks]

BATCH INGEST — Apache Airflow

- Scheduled DAG runs daily at 6 AM IST
- yfinance pulls previous-day OHLCV
- PySpark writes Parquet → MinIO /raw-zone/
- Schema validation before write

STREAM INGEST — Apache Kafka + PySpark SS

- Python producer: simulates tick data at 1 Hz
- Topic: market_ticks (20 stocks, 1-sec interval)
- PySpark Structured Streaming consumer
- foreachBatch → MinIO /raw-zone/stream/

RAW STORAGE — MinIO (local, S3-compatible, free, AGPL-3.0)

[/raw-zone/batch/YYYY-MM-DD/*.parquet (daily OHLCV)] [/raw-zone/stream/ (30-second streaming micro-batches)] [/iceberg-warehouse/ (Iceberg table data and metadata)]

ICEBERG CATALOG — Project Nessie (free, Apache 2.0, git-like data branching)

[Registers all Iceberg tables — Bronze, Silver, Gold] [Supports dev / staging / prod data branches (unique interview talking point)] [Spark connects via: spark.sql.catalog.nessie = org.projectnessie.spark.extensions]

BRONZE LAYER — Apache Iceberg (raw, unmodified, schema-enforced, full history)

[raw_stock_prices (daily OHLCV batch)] [raw_tick_stream (simulated real-time aggregated ticks)] [raw_market_metadata (stock splits, dividends, sector mappings)]

COMPUTE — PySpark 3.5 (local cluster via Docker)

- Reads Iceberg Bronze via Nessie catalog
- Heavy joins, window aggregations
- Writes to Iceberg Silver / Gold tables
- Spark UI: localhost:4040 for debugging

TRANSFORM — dbt Core + dbt-spark adapter (free)

- SQL-first transformation models
- dbt tests: not_null, unique, custom checks
- dbt docs: auto-generated data lineage
- Runs on host machine via pip install dbt-spark

SILVER LAYER — Apache Iceberg (cleaned, validated, business-calendar-aligned)

[clean_prices (deduped, nulls handled, type-cast, trading-day join)] [clean_ticks (windowed 1-min aggregates, anomaly flagged)] [enriched_metadata (sector enrichment, currency-normalised)]

GOLD LAYER — Apache Iceberg (business-ready, denormalised, partitioned by date+sector)

[daily_returns] [volatility_7d_30d] [moving_avg_20_50] [sector_performance] [portfolio_risk_scores]

DATA QUALITY — Great Expectations GX Core (free, Apache 2.0)

[Expectation suites: bronze_suite, silver_suite] [Airflow GreatExpectationsOperator: pipeline halts on DQ failure] [Checks: null rates, schema drift, volume anomaly, value range] [DQ results stored as JSON in MinIO — queryable as Iceberg meta-table]

ORCHESTRATION — Apache Airflow 2.9 (CeleryExecutor, full pipeline DAG with SLA)

[Master DAG: ingest → DQ Bronze → dbt Silver → DQ Silver → dbt Gold → alert] [SLA: pipeline must complete by 8 AM IST — miss triggers Slack/email alert] [Retry policy: 3 retries, exponential backoff on every task]

Apache Superset · Price heatmap by sector · Volatility risk view · DQ monitoring board · SQL-driven, no code	Great Expectations · Data Docs site · Checkpoint history · Anomaly trends · Schema drift alerts	MLflow (Phase 6) · ARIMA/Prophet model · Experiment tracking · Model registry · REST serving endpoint
---	--	---

Why Nessie instead of Hive Metastore (HMS)? HMS was designed for Hive — Iceberg is a bolt-on. Nessie is purpose-built for Iceberg, supports git-like data branching (create a dev branch, test your pipeline changes, merge to main — exactly like code), has a cleaner Docker setup (one container vs HMS's full Hadoop dependency chain), and is the direction the Iceberg community is moving. Every interviewer who knows Iceberg knows Nessie. This choice alone is a 5-minute conversation in an interview.

COMPLETE TECH STACK

Every tool, license, Docker image, and reason

Component	Tool & Version	License	How to Run	Why Not the Alternative
Data Source	yfinance 0.2.x	MIT — Free	pip install yfinance No container needed	Real NSE/BSE OHLCV with no API key or rate limit for daily pulls. Alpha Vantage (500 calls/day free) is too restrictive for historical backfills across 20 stocks.
Orchestration	Apache Airflow 2.9	Apache 2.0 — Free	apache/airflow:2.9.0	You use it at TCS. Interviewers expect it. Prefect and Dagster are excellent alternatives but Airflow has 10x more interview coverage in the Indian DE market. Go deeper on what you know.
Message Queue	Apache Kafka 3.7 + Zookeeper	Apache 2.0 — Free	bitnami/kafka:3.7 bitnami/zookeeper	Industry standard for event streaming. Pulsar is technically superior for some use cases but Kafka is the lingua franca interviewers expect. RabbitMQ is a message broker, not a streaming platform — wrong tool.
Compute	Apache Spark 3.5 (PySpark)	Apache 2.0 — Free	bitnami/spark:3.5.1 (master + 1 worker)	You use it at TCS on Iceberg. Keep this as your core skill. Polars is faster on a single machine but not distributed. Flink is better for streaming but a 6-month learning project alone. Spark covers both batch and streaming.
Table Format	Apache Iceberg 1.5 (JAR bundled with Spark)	Apache 2.0 — Free	iceberg-spark-runtime JAR added to Spark config	You use it at TCS. Vendor-neutral (unlike Delta Lake which is Databricks-first). Hudi has less adoption in Indian fintech hiring. Iceberg is the right choice to deepen rather than diversify.
Iceberg Catalog	Project Nessie 0.80 (REST catalog)	Apache 2.0 — Free	projectnessie/nessie:0.80.0	Purpose-built for Iceberg. Git-like data branching for free. HMS (Hive Metastore) works but needs Hadoop dependencies. SQLite catalog works only for single-process — not realistic.
Object Storage	MinIO	AGPL-3.0 — Free (personal/dev use)	minio/minio:latest	Local S3-compatible API. Iceberg + Spark use S3 API — MinIO lets you develop and test without AWS costs. HDFS is the alternative but overkill for single-machine. LocalStack adds unnecessary complexity.
SQL Transform	dbt Core 1.8 + dbt-spark adapter	Apache 2.0 — Free	pip install dbt-spark (runs on host OS)	dbt Core is permanently free. dbt Cloud is paid — never needed here. Raw Spark SQL works but misses model testing, auto-generated lineage docs, and the dbt DAG — all of which are interview talking points.
Data Quality	Great Expectations GX Core 0.18+	Apache 2.0 — Free	pip install great-expectations	GX Core is free forever. GX Cloud is paid — ignore it. soda-core is a solid free alternative. elementary.data is better for dbt-native DQ. GX wins on Airflow integration and interview recognition.
Visualization	Apache Superset 4.x	Apache 2.0 — Free	apache/superset:4.0.0	You already use it at TCS. SQL-driven — plays to your strength. Metabase is easier but less powerful. Grafana is metric-oriented. Redash works but smaller community. Keep building on what you know.
Experiment Track	MLflow 2.12 (Phase 6 only)	Apache 2.0 — Free	ghcr.io/mlflow/mlflow:v2.12.0	Already on your resume from TCS. Phase 6 addition demonstrates the platform can support ML workloads — a strong differentiator for DE roles at AI-first fintechs. Weights & Biases has paid tiers and less DE-interview coverage.
Metadata DB	PostgreSQL 15	PostgreSQL — Free	postgres:15-alpine	Airflow and Superset both require a relational backend. PostgreSQL is the standard. SQLite works for Airflow in dev but fails under any production-like load. One shared Postgres instance serves both services.
Container Infra	Docker Compose v2	Apache 2.0 — Free	docker-compose.yml All 12 services defined	Kubernetes is production standard but massively overengineered for a one-machine portfolio project. Docker Compose gives you service isolation, networking, volumes — same concepts, fraction of the complexity. Add k8s to your learning list for Year 2.

12-MONTH BUILD TIMELINE

Evening build block 9–11 PM weekdays + Saturday 8 AM–12 PM

MONTH	FOCUS	WHAT TO BUILD	DELIVERABLES	SKILLS GAINED
Month 1	Docker Stack Foundation	• docker-compose.yml: Airflow, MinIO, Spark master+worker, Nessie, PostgreSQL • MinIO bucket structure: /raw-zone/ and /iceberg-warehouse/ • Spark connected to Nessie catalog and MinIO • Test write: create one Iceberg table from PySpark interactively	• docker-compose up starts entire stack • PySpark writes an Iceberg test table to MinIO via Nessie • README with architecture diagram committed to GitHub	• Docker Compose service networking • Spark-Iceberg-Nessie connector config • MinIO S3 API (endpoint, keys)
Month 2	Bronze Layer — Batch Ingest	• Airflow DAG: bronze_ingest_daily.py • yfinance: pulls previous-day OHLCV for 20 NSE stocks • PySpark writes to Iceberg Bronze: raw_stock_prices table • Schema enforcement on write, basic null check before write • Backfill: load 12 months of historical data	• Working DAG runs daily at 6 AM IST without manual trigger • raw_stock_prices has 12 months history, partitioned by date • Iceberg time travel verified: SELECT ... AS OF VERSION 1	• Airflow DAG authoring, scheduling • Iceberg write modes (append vs overwrite) • PySpark schema enforcement on write
Month 3	Silver Layer — dbt Cleaning	• dbt project init with dbt-spark adapter profile • sources.yml pointing to Bronze Iceberg tables via Nessie • Silver model: clean_prices (dedup, nulls, type cast, trading-day calendar join) • dbt tests: not_null, unique, accepted_values on all models • dbt docs generate — HTML lineage committed to repo	• dbt run produces clean_prices Silver Iceberg table • dbt test passes with zero failures on full history • dbt docs lineage shows Bronze → Silver data flow	• dbt model DAG and dependency graph • SQL window functions for deduplication • dbt testing framework — writing custom tests
Month 4	Gold Layer — Aggregations	• dbt Gold models: daily_returns, volatility_7d_30d, moving_avg_20_50 • Sector-level aggregations using NSE sector mapping seed file • Simple risk score: volatility percentile rank within sector • Iceberg partitioning: by date and sector for query performance • Airflow DAG extended to trigger dbt run after Bronze ingest	• Gold Iceberg tables queryable in Superset • dbt lineage graph covers full Bronze → Silver → Gold • Blog post 1 draft: 'Architecture decisions in FinLake'	• Iceberg partitioning strategy • dbt macros for reusable SQL logic • Aggregation window functions at data-warehouse scale
Month 5	Data Quality — Great Expectations	• GX expectations suite for Bronze: null rate < 2%, schema consistency, daily volume range • GX expectations suite for Silver: business rule checks, no negative prices • Airflow GreatExpectationsOperator integration in full pipeline DAG • Pipeline halts (task fails) if any DQ check fails • DQ checkpoint results stored as JSON in MinIO — queryable as Iceberg table	• Pipeline halts cleanly with error message when DQ fails • DQ results form a meta-monitoring layer — 'data about data quality' • Blog post 1 published on Hashnode	• GX Core API: expectations, checkpoints, validators • Airflow failure handling and downstream skipping • Data observability design patterns

Month 6	Full Orchestration + SLA Monitoring	- Master DAG: ingest → DQ Bronze → dbt Silver → DQ Silver → dbt Gold → success alert - SLA: full pipeline must complete by 8 AM IST — SLA miss sends email alert - Retry policy: 3 retries, exponential backoff (2 min, 4 min, 8 min) on every task - Airflow Variables for config (stock list, date ranges) - Backfill strategy: how to replay a missed day without duplicates	- One production-grade DAG runs reliably without manual intervention - SLA miss triggers real alert — tested by deliberately breaking a task - Full pipeline documented in README with troubleshooting guide	- Airflow SLA monitoring and missed SLA callbacks - Idempotent pipeline design (safe to rerun any day) - Incident recovery playbook for data pipelines
Months 7–8	Kafka Streaming Layer	- Python Kafka producer: simulates tick data at 1 Hz for all 20 stocks - PySpark Structured Streaming consumer: Kafka topic → Iceberg Bronze streaming table - Watermarking: 5-minute late-data tolerance - forEachBatch pattern: micro-batch write to Iceberg every 30 seconds - Airflow DAG: daily reconcile between batch Bronze and streaming Bronze tables	- Kafka producer running continuously in Docker - Streaming Bronze Iceberg table updated every 30 seconds — verifiable in MinIO - Reconciliation DAG catches any count discrepancy and logs it	- Kafka producer/consumer API + consumer groups - PySpark Structured Streaming + watermarking - Iceberg streaming writes via forEachBatch - Exactly-once vs at-least-once delivery tradeoffs
Months 9–10	Superset Dashboards + Writing	- Dashboard 1: Stock performance heatmap by NSE sector (Gold layer) - Dashboard 2: Volatility and 30-day risk overview per stock - Dashboard 3: DQ monitoring — daily check pass/fail history 5 Architecture Decision Records (ADRs) committed to docs/architecture/decisions/ - Blog post 2: 'Why I added Kafka streaming to FinLake — and what broke'	3 live dashboards demo-able in under 2 minutes in any interview 2 published blog posts with real architectural reasoning 5 ADR markdown files — the single biggest interview differentiator	- Superset chart types and SQL dataset design - Technical writing: architecture decision records - How to communicate engineering tradeoffs in writing
Months 11–12	MLflow + Open Source Contribution	- MLflow added to docker-compose.yml with PostgreSQL backend - Train ARIMA or Prophet model on Gold layer daily_returns data - MLflow tracking: log params, RMSE/MAE metrics, model artifact - MLflow model registry: register best run as 'production' version - Target: one merged PR to Apache Iceberg, dbt-spark, or Great Expectations	- MLflow UI shows full experiment history with metric comparison - Forecasting model served via MLflow REST endpoint - Your name in a production open-source codebase commit (permanent)	- MLflow tracking API and model registry patterns - Time-series forecasting basics (ARIMA, Prophet) - Open-source contribution workflow: fork → PR → review → merge

```
finlake/
■■■■ docker-compose.yml ← Entire stack. One command to start everything.
■■■■ .env ← Ports, bucket names, passwords. Never commit this.
■■■■ Makefile ← make up | make down | make dbt-run | make test
■■■■ README.md ← Architecture diagram + one-command setup guide
■
■■■■ airflow/dags/
■ ■■■■ bronze_ingest_daily.py ← yfinance → PySpark → Iceberg Bronze
■ ■■■■ dbt_transform.py ← Triggers dbt run for Silver then Gold
■ ■■■■ dq_bronze_check.py ← GX checkpoint on Bronze layer
■ ■■■■ dq_silver_check.py ← GX checkpoint on Silver layer
■ ■■■■ streaming_reconcile.py ← Daily batch vs stream Bronze count check
■ ■■■■ full_pipeline.py ← Master DAG: all above chained with SLA
■
■■■■ spark/jobs/
■ ■■■■ bronze_writer.py ← PySpark batch write to Iceberg Bronze
■ ■■■■ streaming_consumer.py ← PySpark Structured Streaming: Kafka → Iceberg
■
■■■■ dbt/
■ ■■■■ models/staging/ ← sources.yml pointing at Bronze Iceberg tables
■ ■■■■ models/silver/ ← clean_prices.sql clean_ticks.sql
■ ■■■■ models/gold/ ← daily_returns.sql volatility.sql sector_perf.sql
■ ■■■■ tests/ ← Custom dbt tests beyond built-in
■
■■■■ kafka/producers/
■ ■■■■ tick_simulator.py ← Produces simulated tick data at 1 Hz for 20 stocks
■
■■■■ great_expectations/
■ ■■■■ expectations/ ← bronze_suite.json silver_suite.json
■ ■■■■ checkpoints/ ← Checkpoint configs used by Airflow operator
■
■■■■ superset/dashboards/ ← Exported JSON for reproducible dashboard import
■
■■■■ mlflow/experiments/
■ ■■■■ price_forecasting.py ← ARIMA/Prophet training + MLflow logging script
■
■■■■ docs/architecture/decisions/ ← Architecture Decision Records (ADRs)
■■■■ ADR-001-why-iceberg-not-delta.md
■■■■ ADR-002-why-nessie-not-hms.md
■■■■ ADR-003-why-kafka-not-batch-only.md
■■■■ ADR-004-why-dbt-not-raw-spark-sql.md
■■■■ ADR-005-why-gx-not-custom-dq-checks.md
```

ARCHITECTURE DECISION RECORDS (ADRs)

Write one per major tool choice — biggest interview differentiator

ADR	DECISION	REASONING
ADR-001	Why Apache Iceberg and not Delta Lake	Delta Lake is excellent but Databricks-first. Iceberg is vendor-neutral, works with any Spark/Trino/Flink setup, and is the format your TCS project uses — deepening is smarter than diversifying. Hudi has lower adoption in Indian fintech hiring. Iceberg also has superior schema evolution and hidden partitioning.
ADR-002	Why Project Nessie and not Hive Metastore	HMS was designed for Hive — Iceberg is an afterthought in HMS. Nessie is purpose-built for Iceberg, supports git-like data branching (create a dev branch, test pipeline changes, merge to main without affecting production data), and needs just one Docker container vs HMS's Hadoop dependency chain.
ADR-003	Why add Kafka streaming and not stay batch-only	Batch-only pipelines are sufficient for daily reporting. But senior DE roles at fintechs involve real-time fraud detection, live inventory, and tick-level analytics. Adding Kafka demonstrates you understand the streaming vs batch tradeoff and can implement both. The reconciliation DAG teaches late-data handling — a real production problem that interview questions are built around.

ADR-004	Why dbt and not raw Spark SQL for transformations	Raw Spark SQL works but gives you no testing, no auto-generated lineage documentation, no model DAG, and no dbt test framework. dbt forces you to write testable, documented SQL — the same discipline senior DEs enforce in production. dbt docs generate produces a lineage graph you can show in an interview with one URL. Spark SQL produces nothing comparable.
ADR-005	Why Great Expectations and not custom DQ checks	Custom checks work but are one-off scripts with no standardised output format, no expectation store, and no Airflow operator. GX Core gives you: a reusable expectation suite, a Data Docs site, a native Airflow operator, and JSON-format results you can store and query as an Iceberg meta-table. The last point — DQ results as a queryable dataset — is a design pattern interviewers ask about.

INTERVIEW QUESTIONS THIS PROJECT LETS YOU ANSWER

These are actual questions from senior DE screens at
Indian fintechs

Q#	QUESTION	HOW TO ANSWER IT
Q1	Walk me through your data platform architecture.	Start at the source layer. Describe the batch and streaming ingest paths. Explain why they both write to Bronze and how the reconciliation DAG catches discrepancies. Mention Nessie catalog and why. End at Gold and Superset.
Q2	Why did you choose Iceberg? What does it give you that a raw Parquet file doesn't?	ACID transactions, snapshot isolation, time travel (SELECT AS OF VERSION N), schema evolution without rewrite, hidden partitioning, and metadata-driven query planning. These are not features you can bolt onto Parquet.
Q3	What is copy-on-write vs merge-on-read in Iceberg and which did you use?	CoW rewrites full data files on update/delete — better read performance, higher write cost. MoR writes delete files separately — cheaper writes, more expensive reads (merge at query time). For append-only financial data (new rows daily), CoW (the Iceberg default) is correct. If you had frequent updates, MoR would be better.
Q4	Your Kafka consumer is falling behind — what do you do?	First check consumer group lag via Kafka UI or kafka-consumer-groups.sh. If lag is growing: (1) increase Spark micro-batch trigger interval to process larger batches, (2) increase consumer parallelism (more partitions = more tasks), (3) check if the bottleneck is Iceberg write latency or Kafka read latency. Monitor via Spark UI streaming tab.
Q5	How does your pipeline handle a DQ failure? What happens to downstream tasks?	The GX checkpoint Airflow operator raises AirflowException on failure. Airflow marks that task as failed. All downstream tasks are automatically skipped. The SLA miss alert fires because the downstream dbt tasks never ran. The on-failure callback sends a Slack/email notification with the specific expectation that failed.
Q6	Your dbt run produced wrong numbers in the Gold layer. How do you debug it?	Step 1: dbt test on Silver — find the first failing test. Step 2: run the Silver model in isolation with dbt run -s model_name. Step 3: query the Bronze source directly to see if the issue is upstream. Step 4: use Iceberg time travel to compare today's Silver table with yesterday's to isolate the change. Step 5: fix the model, dbt run, dbt test, commit.
Q7	What is Nessie and why does it matter for Iceberg?	Nessie is a transactional catalog for Iceberg with git-like branching. You can create a dev branch, run your pipeline changes against it, verify the output, and merge to main — without affecting the production data branch at all. This is the same workflow as code branching, applied to data. HMS doesn't support this.
Q8	How would you make this platform handle 10x the data volume?	(1) Horizontal Spark scaling: add more worker nodes to Docker Compose or move to Kubernetes. (2) Iceberg partitioning: ensure partition pruning is working — check Spark UI for files scanned vs files skipped. (3) Compaction: schedule Iceberg compaction to merge small files produced by streaming writes. (4) Kafka partitions: increase topic partitions to allow more parallel consumer tasks. (5) dbt: move from local Spark to a dedicated Spark cluster or Trino for query serving.

YOUR FIRST ACTION — tonight, not next week:

Open terminal. Run:

```
mkdir finlake && cd finlake && git init
touch README.md docker-compose.yml Makefile .env
mkdir -p airflow/dags spark/jobs dbt/models/{staging,silver,gold} kafka/producers
great_expectations/{expectations,checkpoints} superset/dashboards mlflow/experiments
docs/architecture/decisions
echo '# FinLake — BFSI Data Platform' > README.md
git add . && git commit -m 'init: project scaffold'
```

Push to GitHub. Your project now exists with a timestamp. Everything else follows from that commit.